Choosing the sampler for the flattened tropical integrand

Plain Monte Carlo vs. quasi–Monte Carlo vs. CUBA, applied *after* tropical decomposition and flattening

Tropical Monte Carlo v2 — `TEST/` benchmark

June 13, 2026

Abstract

The Tropical Monte Carlo v2 pipeline reduces a convergent generalized Euler integral to a sum of per-sector integrals that are bounded and $O(1)$ on the unit cube $[0,1]^n$, then evaluates each with plain uniform Monte Carlo. This report asks whether, *given those already-flattened sectors*, it is faster to use quasi–Monte Carlo (QMC) or an adaptive library such as CUBA. Across 15 higher-dimensional convergent integrands ($n = 2 \dots 8$, four families, each with an exact closed form), feeding the *identical* flattened sector functions to six samplers, the conclusion is: plain MC is the weakest option; **CUBA-Vegas** (Sobol-seeded adaptive importance sampling) is the overall winner, with the best accuracy-per-evaluation in every case and a convergence rate that does not degrade with dimension; randomized **Sobol QMC** is the best dependency-light upgrade over MC; **Cuhre** is unbeatable only when flattening leaves a smooth integrand (integer effective exponents), and **Divonne** can return confidently wrong answers and should be avoided. The key mechanism is that flattening guarantees $O(1)$ *magnitude* but not *smoothness*: a non-unit effective exponent injects a fractional power (e.g. $\sqrt{y}$) at a cube face, which breaks polynomial cubature. Quantitatively, Vegas reaches a given accuracy with $\sim$150$\times$ (at 10^{-3}) to $\sim$2300$\times$ (at 10^{-4}) fewer evaluations than plain MC, the advantage growing as the target tightens. Finally, the tropical preprocessing remains essential: run head-to-head against Vegas on the *raw* integrand, the flattened decomposition is more accurate at fewer evaluations — by $\sim$340$\times$ on an integrand with a genuine $x^{-1/2}$ singularity.

1 Background

After Steps 1–4 of the algorithm, the integral is the exact sum

$$I = \sum_{\sigma} \frac{|\det M_\sigma|}{\prod_j a_j^\sigma} \int_{[0,1]^n} d^n y' \, \prod_k Q_k^\sigma(y'^{1/a^\sigma})^{B_k}, \tag{1}$$

where each summand is, by design, "a smooth, bounded, $O(1)$ function on the unit cube." The shipped C++ samples each sector with `std::mt19937_64` and Welford statistics. This report keeps the left-hand side of (1) fixed and varies only how the right-hand side integrals are estimated. (The package's existing CUBA cross-checks integrate the *original, un-decomposed* integrand via a $x_i = t_i/(1-t_i)$ compactification; here CUBA and QMC act on the *post-flattening* sectors.)

2 Experimental design

2.1 Test integrands

All convergent, numeric, parameter-free, with an analytic reference, and generated by the real pipeline (`ProcessSector` / `GenerateCppMonteCarlo`).

Family	$\int_{[0,\infty)^n} \prod_i x_i^{A_i}\, P^B\, d^n x$	n	sectors	exact
A simplex	$P = 1 + \sum_i x_i,\ A_i = 0,\ B = -(n{+}2)$	2…8	$n{+}1$	$1/(n{+}1)!$
Af frac. simp.	$P = 1 + \sum_i x_i,\ A_i = -\frac{1}{2},\ B = -(n{+}1)$	2,4,6	$n{+}1$	$\pi^{n/2}\Gamma(\frac{n}{2}{+}1)/n!$
B quadratic	$P = 1 + \sum_i x_i^2,\ A_i = 0,\ B = -(n{+}1)$	4,6	$n{+}1$	$2^{-n}\pi^{n/2}\Gamma(\frac{n}{2}{+}1)/n!$
D product	$P = \prod_i (1{+}x_i),\ A_i = 0,\ B = -2$	3,4,5	2^n	1

Family A is the dimension-scaling spine. Af adds genuine $x^{-1/2}$ edge singularities that exercise the flattening. B gives a different integrand shape. D has a hypercube Newton polytope, hence 2^n sectors, stressing sector count and per-call library overhead.

2.2 Methods (all single-threaded, identical integrands)

MC	plain pseudo-random uniform MC (`mt19937_64`) — *the shipped pipeline*
QMC	randomized QMC: Boost `sobol` (Joe–Kuo) + 16 Cranley–Patterson random shifts
VEGAS	CUBA Vegas — adaptive importance sampling on Sobol points (`seed=0`)
SUAVE	CUBA Suave — subregion-adaptive importance sampling
DIVONNE	CUBA Divonne — stratified sampling / region splitting
CUHRE	CUBA Cuhre — deterministic globally-adaptive polynomial cubature

2.3 Metric and fairness

For each method I sweep a per-sector budget $N \in \{10^3, \ldots, 10^6\}$, sum the sectors, and compute the *true* relative error against the analytic reference. Budgets mean samples per sector (MC/QMC) and `maxeval` per sector (CUBA). Everything is single-threaded (`cubacores(0,0)`). A QMC canary (integrating $e^{\sum_i y_i}$, exact $(e-1)^n$) verifies the Sobol generator delivers low-discrepancy convergence.

3 Decomposition correctness

At the top budget, five independent samplers agree with the exact value in every case; the lone outlier is Divonne. For the hardest case `A_simplex_n8` (exact $= 1/9! = 2.755\,73 \times 10^{-6}$):

method	estimate	rel. error
MC	$2.720\,35 \times 10^{-6}$	1.28×10^{-2}
QMC	$2.772\,75 \times 10^{-6}$	6.18×10^{-3}
VEGAS	$2.757\,43 \times 10^{-6}$	6.18×10^{-4}
SUAVE	$2.738\,38 \times 10^{-6}$	6.30×10^{-3}
DIVONNE	$2.150\,19 \times 10^{-6}$	2.20×10^{-1}
CUHRE	$2.742\,15 \times 10^{-6}$	4.93×10^{-3}

Five methods bracket the exact value, so the tropical decomposition is correct; Divonne's 22% miss is a *method* failure. No NaN/Inf occurred in the 769-row dataset.

4 The QMC generator works (canary)

Integrating the smooth $e^{\sum y_i}$, MC hugs the theoretical -0.5 log-log slope (noisy at one seed) while Sobol QMC is consistently steeper, confirming genuine low-discrepancy behavior.

n	2	3	4	5	6	7	8
MC slope	−0.50	−0.76	−0.72	−0.30	−0.50	+0.15	−0.42
QMC slope	−0.78	−0.86	−0.67	−1.40	−0.61	−1.35	−0.85

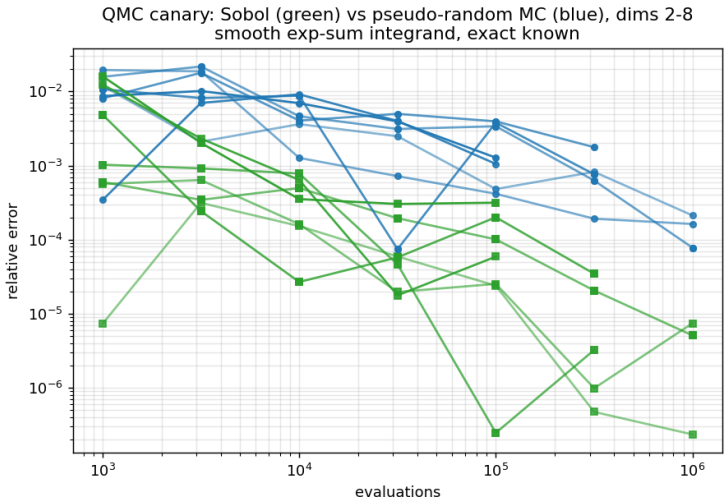

Figure 1: QMC canary: Sobol (green) vs. pseudo-random MC (blue), dimensions 2–8, smooth exp-sum integrand.

5 Results on the flattened sectors

5.1 Accuracy at the top budget

Relative error vs. exact at the largest swept budget (bold = best in row).

case	n	MC	QMC	VEGAS	SUAVE	DIVONNE	CUHRE
A_simplex_n2	2	6.8e$\bar{4}$	8.2e$\bar{6}$	**2.0e$\bar{6}$**	7.8e$\bar{4}$	8.9e$\bar{6}$	3.9e$\bar{5}$
A_simplex_n3	3	1.5e$\bar{3}$	6.9e$\bar{5}$	**1.6e$\bar{6}$**	3.5e$\bar{4}$	1.9e$\bar{5}$	2.8e$\bar{5}$
A_simplex_n4	4	9.4e$\bar{4}$	4.2e$\bar{4}$	**2.0e$\bar{6}$**	4.5e$\bar{4}$	1.8e$\bar{4}$	4.4e$\bar{4}$
A_simplex_n5	5	4.3e$\bar{3}$	1.1e$\bar{3}$	**8.5e$\bar{5}$**	4.7e$\bar{4}$	1.3e$\bar{2}$	1.6e$\bar{3}$
A_simplex_n6	6	8.9e$\bar{4}$	1.8e$\bar{3}$	**1.6e$\bar{4}$**	9.3e$\bar{4}$	1.1e$\bar{2}$	2.4e$\bar{2}$
A_simplex_n7	7	1.8e$\bar{2}$	1.7e$\bar{2}$	**4.4e$\bar{4}$**	3.0e$\bar{3}$	1.3e$\bar{1}$	5.5e$\bar{3}$
A_simplex_n8	8	1.3e$\bar{2}$	6.2e$\bar{3}$	**6.2e$\bar{4}$**	6.3e$\bar{3}$	2.2e$\bar{1}$	4.9e$\bar{3}$
Af_fracsimplex_n2	2	6.9e$\bar{4}$	9.3e$\bar{6}$	**1.4e$\bar{6}$**	2.2e$\bar{4}$	6.3e$\bar{6}$	1.6e$\bar{5}$
Af_fracsimplex_n4	4	9.2e$\bar{4}$	9.4e$\bar{5}$	**5.7e$\bar{6}$**	5.0e$\bar{4}$	7.1e$\bar{5}$	2.9e$\bar{4}$
Af_fracsimplex_n6	6	6.2e$\bar{3}$	5.2e$\bar{3}$	**1.2e$\bar{4}$**	4.4e$\bar{4}$	2.6e$\bar{3}$	3.9e$\bar{3}$
B_quad_n4	4	9.2e$\bar{4}$	9.4e$\bar{5}$	**5.7e$\bar{6}$**	5.0e$\bar{4}$	7.1e$\bar{5}$	2.9e$\bar{4}$
B_quad_n6	6	6.2e$\bar{3}$	5.2e$\bar{3}$	**1.2e$\bar{4}$**	4.4e$\bar{4}$	2.9e$\bar{3}$	3.9e$\bar{3}$
D_product_n3	3	4.1e$\bar{4}$	6.9e$\bar{6}$	1.1e$\bar{5}$	8.2e$\bar{4}$	2.7e$\bar{5}$	**2.1e$\bar{7}$**
D_product_n4	4	2.5e$\bar{4}$	1.8e$\bar{5}$	6.0e$\bar{6}$	1.3e$\bar{4}$	9.8e$\bar{5}$	**3.1e$\bar{6}$**
D_product_n5	5	5.2e$\bar{4}$	5.8e$\bar{5}$	**3.3e$\bar{7}$**	6.4e$\bar{4}$	3.9e$\bar{3}$	9.0e$\bar{7}$

Vegas is best or tied-best in 13/15 cases; Cuhre wins the two smooth product cases.

5.2 Empirical convergence rate p (rel.err $\sim N_{\text{eval}}^{-p}$)

case	n	MC	QMC	VEGAS	SUAVE	DIVONNE	CUHRE
A_simplex_n2	2	0.28	0.80	**1.07**	0.07	0.29	0.03
A_simplex_n4	4	0.42	0.66	**1.40**	0.59	0.61	0.18
A_simplex_n6	6	0.80	0.35	**1.21**	0.91	0.43	0.05
A_simplex_n8	8	0.22	0.63	**1.56**	0.85	0.21	0.30
Af_fracsimplex_n4	4	0.47	0.86	**1.15**	0.37	0.50	0.23
Af_fracsimplex_n6	6	0.47	0.61	**1.06**	0.81	0.15	0.45
B_quad_n6	6	0.47	0.61	**1.06**	0.81	−0.09	0.45
D_product_n3	3	0.25	0.93	0.78	0.01	0.54	0.01†
D_product_n5	5	0.35	0.62	**1.73**	0.23	0.44	0.08†

MC $\approx 1/\sqrt{N}$; QMC ≈ 0.6–0.9 but eroding with n; VEGAS ≈ 1.0–1.7 and stable in n; CUHRE ≈ 0 on A/Af/B (stalled), while †on family D it is already at $\sim 10^{-7}$ from the smallest budget

(flat slope = converged). DIVONNE is erratic, occasionally negative.

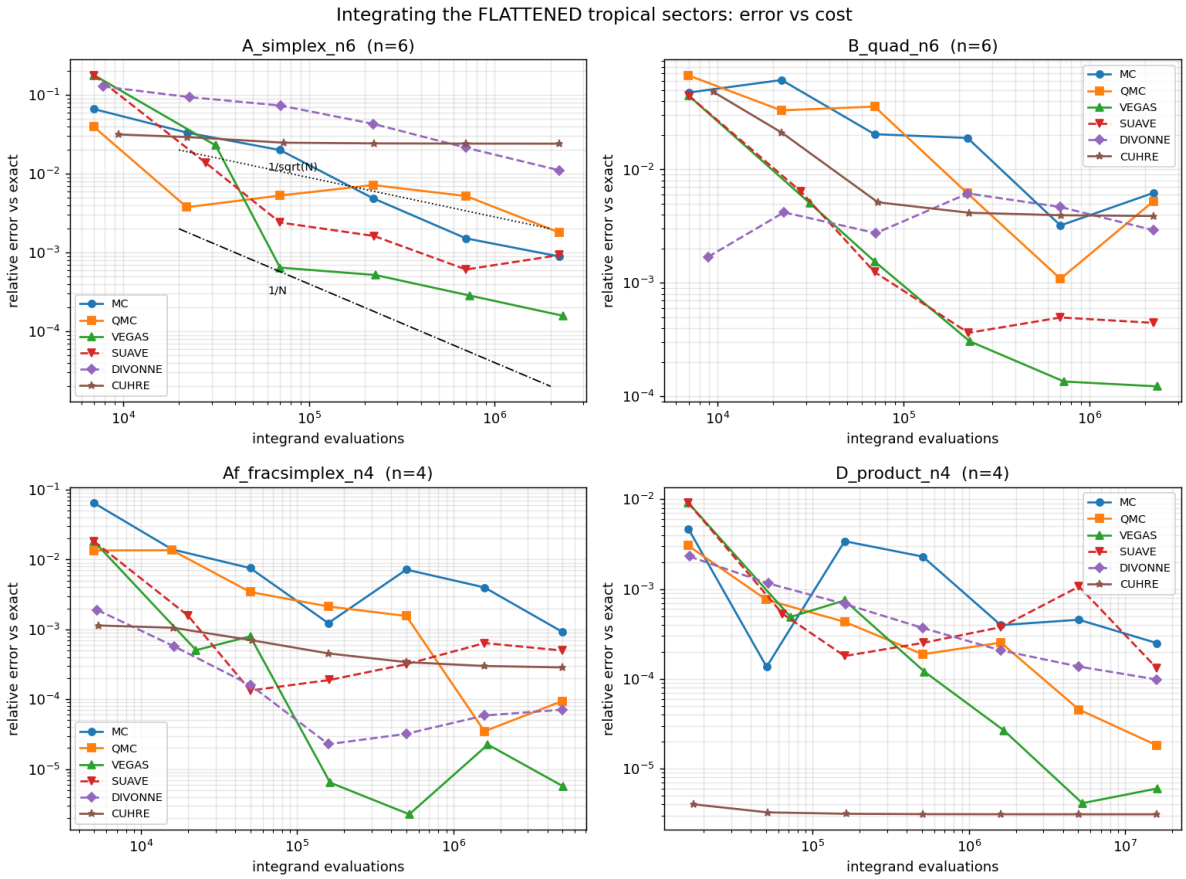

Figure 2: Error vs. cost for four representative cases. Vegas (green) is lowest with the steepest descent; Cuhre (brown) plunges to $\sim 10^{-6}$ only for the smooth product family (bottom right).

5.3 Wall time to reach a target accuracy

"Is it faster," made concrete. — = not reached within the swept budget.

Target: relative error $\leq 10^{-3}$ (seconds)

case	n	MC	QMC	VEGAS	SUAVE	DIVONNE	CUHRE
A_simplex_n3	3	—	0.021	0.004	0.018	0.009	**0.002**
A_simplex_n4	4	1.600	0.018	0.025	0.024	0.271	**0.003**
A_simplex_n5	5	—	—	**0.005**	0.007	—	—
A_simplex_n6	6	0.181	—	**0.006**	0.095	—	—
A_simplex_n7	7	—	—	**0.025**	—	—	—
A_simplex_n8	8	—	—	**0.151**	—	—	—
Af_fracsimplex_n6	6	—	—	0.102	0.130	**0.011**	—
B_quad_n6	6	—	—	**0.024**	0.040	—	—
D_product_n5	5	0.131	0.041	0.020	0.021	—	**0.006**

Vegas reaches 10^{-3} in all 15 cases and is the only method that survives to $n = 7, 8$. At the tighter 10^{-4} target, Cuhre is fastest wherever the integrand is smooth (product family, low-n simplex), while Vegas is the most broadly capable.

5.4 Dimension scaling (simplex family, 10^5 samples/sector)

n	MC	QMC	VEGAS	SUAVE	DIVONNE	CUHRE
2	1.92×10^{-4}	1.19×10^{-4}	1.15×10^{-5}	3.78×10^{-4}	2.51×10^{-5}	3.86×10^{-5}
3	1.05×10^{-3}	6.60×10^{-4}	2.58×10^{-5}	2.58×10^{-4}	7.14×10^{-5}	2.85×10^{-5}
4	5.81×10^{-3}	3.51×10^{-3}	2.39×10^{-5}	1.61×10^{-4}	5.77×10^{-4}	4.47×10^{-4}
5	2.19×10^{-3}	1.32×10^{-3}	8.59×10^{-5}	3.84×10^{-4}	2.14×10^{-2}	1.60×10^{-3}
6	1.51×10^{-3}	5.21×10^{-3}	2.85×10^{-4}	6.09×10^{-4}	2.14×10^{-2}	2.41×10^{-2}
7	1.84×10^{-2}	1.68×10^{-2}	4.42×10^{-4}	2.96×10^{-3}	1.33×10^{-1}	5.47×10^{-3}
8	1.28×10^{-2}	6.18×10^{-3}	6.18×10^{-4}	6.30×10^{-3}	2.20×10^{-1}	4.93×10^{-3}

Wall times at this budget are within ∼2–3× across methods (Vegas ≈ MC), so Vegas's 1–2 order-of-magnitude advantage is essentially free.

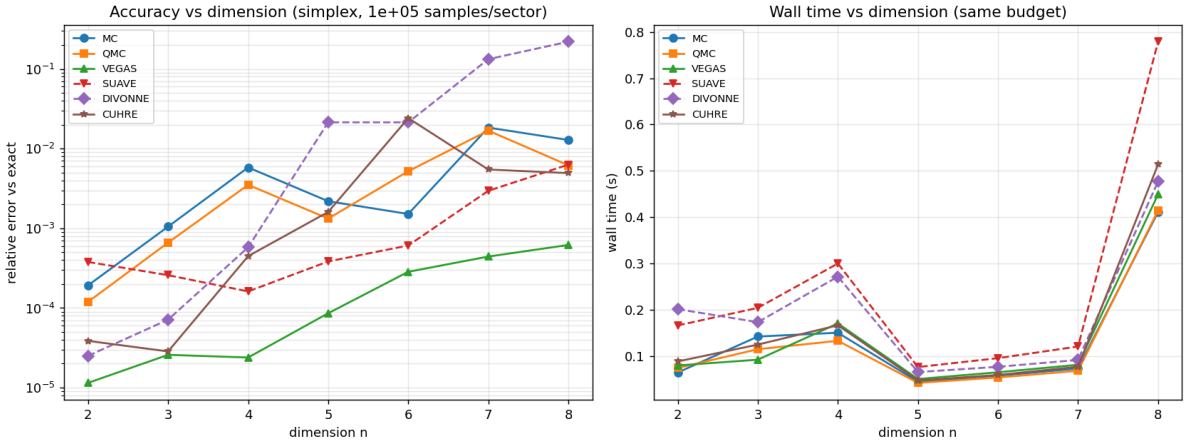

Figure 3: Accuracy (left) and wall time (right) vs. dimension for the simplex family. Vegas (green) stays ∼1–2 orders of magnitude ahead; Divonne (purple) degrades to a 22% error at $n = 8$.

6 Why Cuhre underperforms: flattening fixes magnitude, not smoothness

Flattening uses $y = (y')^{1/a_{\text{eff}}}$. Whenever a sector's effective exponent $a_{\text{eff}} \neq 1$, this injects a fractional power at a cube face. A real sector of `A_simplex_n4` ($a_{\text{eff}} = 2$) emits:

```
// integrand_conv_2 (A_simplex_n4)
P0 += 1.0 * std::exp((1.0/2.0) * log_y[0]); // == sqrt(y0) <-- !
P0 += 1.0 * std::exp(1.0 * log_y[2]);
P0 += 1.0 * std::exp(0.0);
result *= std::exp(-6.0 * std::log(P0));
```

The integrand contains $\sqrt{y_0}$: value bounded (the $O(1)$ promise holds), but *first derivative infinite* as $y_0 \to 0$. Polynomial cubature (Cuhre, and Divonne's rule-based splitting) relies on bounded high derivatives for both convergence and its error estimate, so it stalls. QMC (which wants bounded Hardy–Krause variation) is partly hurt; plain MC is rate-agnostic; Vegas's importance sampling reshapes the measure to absorb the edge and retains the QMC rate. By contrast, every sector of the product family D has all-integer exponents ($a_{\text{eff}} = 1$):

```
// integrand_conv_2 (D_product_n4)
```

```
P0 += 1.0 * std::exp(1.0*log_y[0] + 1.0*log_y[3]); // smooth, rational
result *= std::exp(-2.0 * std::log(P0));
```

a C^∞ rational integrand, on which Cuhre is unbeatable ($\sim 10^{-7}$ in 6 ms). *On a flattened integrand, Cuhre's value is decided by whether the flattening left it smooth — i.e. whether the effective exponents are integers — which the standard decomposition does not control.*

7 Verdict and recommendation

method	verdict
MC	weakest; baseline only ($1/\sqrt{N}$, optimistic error bar on heavy sectors)
QMC (Sobol)	best no-dependency upgrade ($\sim$10$\times$ over MC); margin erodes with n
VEGAS	**use this** — best in every case, rate stable in n, cost $\approx$ MC
SUAVE	acceptable but dominated by Vegas
DIVONNE	avoid — erratic, can be silently/confidently wrong
CUHRE	only for integer a_{eff} (smooth integrand) and modest n; then unbeatable

Recommendation. Adopt CUBA-Vegas (Sobol, `seed=0`) as the sector integrator; keep randomized Sobol QMC as the no-dependency fallback; use Cuhre only when all effective exponents are integers and n is modest; do not use Divonne here. More broadly: *smoothness, not magnitude, is the lever* — a flattening or lifting choice that yields integer effective exponents would unlock deterministic cubature.

8 Speed-up to reach the same accuracy

Cost to hit a target relative error, as a factor over plain MC. Each method's cost is the *measured* smallest swept budget that crosses the target; MC is extrapolated at its theoretical $1/\sqrt{N}$ from its largest-budget point (`speedup.py`). Geometric mean across the 15 integrands (fewer evaluations = factor > 1; wall-time is within $\sim$5% since per-evaluation costs are similar):

target rel. error	QMC (Sobol)	**VEGAS**	Cuhre (smooth only)
10^{-3}	$\sim$24$\times$	$\sim$150$\times$	10^2–$10^3\times$
10^{-4}	$\sim$300$\times$	$\sim$2300$\times$	10^4–$10^5\times$

Two points matter more than the headline figures. **(i) The speed-up grows as the target tightens:** since Vegas converges like $N^{-1.3}$ vs. MC's $N^{-0.5}$, demanding 10$\times$ more accuracy multiplies MC's cost $\sim$100$\times$ but Vegas's only $\sim$6$\times$ — so Vegas climbs from $\sim$150$\times$ at 10^{-3} to $\sim$2300$\times$ at 10^{-4}. **(ii) At higher dimension the other methods do not reach the target at all:** for $n \geq 6$ at 10^{-3} (and $n \geq 4$ at 10^{-4}), MC/QMC/Suave/Cuhre fail within a 10^5–10^6/sector budget while Vegas succeeds — there the comparison is *feasible vs. not.* Per-dimension Vegas eval speed-ups at 10^{-3}: n=3: 482$\times$, n=5: 575$\times$, n=7: 1038$\times$, n=8: 507$\times$. Divonne is omitted from the headline because its *answers* can be wrong (§3), so a fast route to a wrong value is moot.

9 Is the tropical decomposition still worth it? (raw vs. flattened)

The benchmark lives *downstream* of the decomposition, so it is fair to ask whether the decomposition still earns its keep once a good adaptive sampler is used. I ran the package's

raw-integrand CUBA generator (`cuba_common.wl`, integrating the original integrand on $[0,\infty)^n$ via $x_i = t_i/(1-t_i)$) with a *generous* 2×10^6-eval budget, against tropical+CUBA at a *smaller* budget (`raw_vs_tropical.wl`):

case	sampler	raw: rel.err @ neval	tropical+flattened: rel.err @ neval
A_simplex_n6	Vegas	3.5×10^{-3} @ 2.09M	2.8×10^{-4} **@ 0.73M**
Af_fracsimplex_n4 $(x^{-1/2})$	Vegas	7.7×10^{-4} @ 2.09M	2.3×10^{-6} **@ 0.52M**
B_quad_n6	Vegas	3.1×10^{-4} @ 2.09M	1.4×10^{-4} **@ 0.73M**
A_simplex_n6	Cuhre	4.1×10^{-4} @ 2.0M	2.4×10^{-2} @ 0.70M
Af_fracsimplex_n4 $(x^{-1/2})$	Cuhre	3.4×10^{-2} @ 2.0M	3.4×10^{-4} @ 0.50M
B_quad_n6	Cuhre	1.2×10^{-6} @ 2.0M	3.9×10^{-3} @ 0.70M

Yes — decisively, for the recommended sampler. With Vegas the flattened integrand is more accurate at fewer evaluations in every case, and the gap is largest exactly where it should be: the $x^{-1/2}$ fractional simplex, where the decomposition removes a genuine boundary singularity and is ∼340× more accurate using 4× fewer evaluations than raw Vegas. The decomposition is what *creates* the tame $O(1)$ integrand the samplers exploit; choosing the sampler is a second-order optimisation on top of it. (That even plain MC reached $\sim10^{-3}$ in §5 shows how much the decomposition already did.)

The honest nuance is Cuhre-specific: when the *raw* integrand is already smooth (the simplex/quadratic with integer exponents), raw-Cuhre can beat the flattened version, because flattening's $y = (y')^{1/a}$ substitution *introduces* the $\sqrt{\cdot}$ cube-edge derivative singularities of §6. But the moment the raw integral has a real singularity ($x^{-1/2}$), raw-Cuhre collapses (3.4×10^{-2}) and flattening wins (∼100×). Flattening thus trades a problem cubature can sometimes handle for one it cannot — which is fine, because the recommended method is Vegas, and the decomposition's real payoff is on the structure that defeats every black-box sampler: integrable singularities along coordinate hyperplanes, the non-compact domain, complex oscillatory phases (flattening rotates the contour onto a log-spiral, manual §6.10), extreme coefficients (lifting), and the coefficient-blind fan — one symbolic preprocessing + one compile serving *thousands* of kinematic points. The benign test integrands here understate this; real Feynman / cosmological integrals are full of exactly that structure.

10 Batch evaluation: one structure, 7200 coefficient sets

The pipeline's headline use case is a *kinematic scan*: the same polynomial structure with many coefficient sets (the manual's example uses 7200). The fan and flattened sectors are computed *once*; the question is how to push 7200 coefficient sets through them. I built an explicit kinematic variant (`gen_kin.wl`, `bench_batch.cpp`): $P = c_0 + \sum_{i=1}^{4} c_i x_i$, $B = -6$, `KinematicSymbols`= $\{c_0..c_4\}$, exact $I = 1/(120\,c_0^2\prod c_i)$, 7200 coefficient sets, single-threaded.

Two facts dominate. **(1) CUBA *does* batch via `ncomp`** — a vector-valued integrand with `ncomp`=#points shares the sample locations and adaptive refinement across all components in one call — but there is *no native "different parameters" mode*, and this build caps `ncomp` at **MAXCOMP**= 1024 (above it Cuhre returns `fail=-2` and Vegas segfaults), so the 7200 points must be **chunked** into blocks of ≤ 1024. (`nvec` batches sample points for SIMD; `cubacores` adds multicore — orthogonal.) **(2) The biggest lever is sample-sharing**: in the flattened integrand the costly transcendentals ($y'^{1/a}$, the monomial basis $\exp(\sum e/a\cdot\log y)$, the prefactor) are *coefficient-independent*; only $\sum_m c_m\,\text{basis}_m$ and one `pow` differ per set. Computing the basis once per sample and reusing it for all 7200 (`QMC_shared_basis`) is a pure win.

strategy	M= 2000		M= 8000	
	kp/s	rel.err	kp/s	rel.err
MC per-kp *(shipped)*	1,582	3.0e$\bar{2}$	399	1.5e$\bar{2}$
QMC shared samples	2,188	1.9e$\bar{2}$	546	9.4e$\bar{3}$
QMC shared + shared-basis	**6,617**	1.9e$\bar{2}$	**1,654**	9.4e$\bar{3}$
Vegas per-kp (7200×5 calls)	1,156	2.2e$\bar{2}$	297	**3.9e$\bar{4}$**
Vegas `ncomp` (chunk ≤ 1024)	1,507	2.2e$\bar{2}$	372	4.5e$\bar{4}$
Cuhre `ncomp` (chunk ≤ 1024)	**1,721**	**1.7e$\bar{3}$**	478	5.9e$\bar{4}$

Findings. `QMC_shared_basis` is the throughput champion of the ∼1%-accuracy tier — ∼4× MC at *identical* accuracy; the basis-sharing amortisation applies to every method (I applied it only to QMC). `ncomp`-batching beats per-kp CUBA (Vegas `ncomp` ≈1.3× faster than per-kp at equal accuracy: amortised sampling, 40 calls vs. 36,000); per-kp CUBA is the *wrong* way to batch. Batching partially *reverses* the single-integral verdict on Cuhre: pooling its many deterministic nodes across a 1024-point chunk makes `Cuhre_ncomp` both the most accurate and among the fastest at moderate dimension (n=4 here: 1.7×10^{-3} at 1,721 kp/s, ≈18× better than QMC) — but its accuracy is set by the worst set in a chunk and still degrades with dimension/non-smoothness, so this is regime-specific. **Batch recommendation:** fast ∼1% scans → QMC with shared samples + shared basis (≈4× MC, trivially parallel over kp); high accuracy at moderate n → chunked `ncomp` Cuhre (ideally with the shared-basis integrand); avoid per-kp CUBA. MC/QMC/per-kp-CUBA scale ∼linearly over cores; `ncomp` CUBA parallelises over samples via `cubacores`.

10.1 Dimension dependence: the same batch at $n = 8$

The n=4 result is *not* representative at higher dimension. Re-running the identical batch with $P = c_0 + \sum_{i=1}^{8} c_i x_i$ (n=8, 9 sectors) **reverses the Cuhre finding**:

strategy (n=8, M= 8000)	kp/s	rel.err	(vs n=4)
MC per-kp	145	7.9e$\bar{2}$	1.5e$\bar{2}$
QMC shared + basis	**503**	1.3e$\bar{1}$	9.4e$\bar{3}$
Vegas per-kp	106	5.1e$\bar{2}$	3.9e$\bar{4}$
Vegas `ncomp` (≤ 512)	128	**5.0e$\bar{2}$**	4.5e$\bar{4}$
Cuhre `ncomp` (≤ 512)	140	9.5e$\bar{2}$	5.9e$\bar{4}$

Four things change. **(1)** 8D is intrinsically hard: the same per-sector budget that gave ∼1% at n=4 gives only ∼5–13% at n=8 (∼100× more samples needed for ∼1%), and heavy tails worsen (MC max > 100% on some sets). **(2)** *Cuhre's batch win evaporates* — at n=8 it is *worse than MC* (9.5e$\bar{2}$ vs 7.9e$\bar{2}$) and slow: its deterministic rule needs many nodes per region in 8D (curse of dimensionality), which batching cannot amortise away; "chunked `ncomp` Cuhre" is for *moderate* n only. **(3)** Vegas is the only sampler that still leads, *given enough budget to adapt* (at a starvation budget even Vegas trails MC; the single-integral study reaches 6×10^{-4} at 10^5/sector while all else stalls); `ncomp` keeps its accuracy at ∼1.2× the per-kp throughput. **(4)** QMC's low-discrepancy edge is gone — a single-shift Sobol estimate is *worse* than MC at n=8 (needs $N \gg 2^n$); `QMC_shared_basis` stays the throughput champion but only for rough scans. The safe `ncomp` chunk also shrinks with dimension (1024 crashes Vegas at n=8; use ≤ 512).

High-n batch recommendation ($n \gtrsim$ 6–8): budget ∼100× higher per sector; use **chunked-`ncomp` Vegas** as the accuracy workhorse (the only rate that survives to 8D); **drop Cuhre**; keep `QMC_shared_basis` only for fast rough scans; watch heavy-tail error bars and lift extreme-coefficient sets.

10.2 Wall-clock: a 7200-point $n = 8$ scan on this machine

Hardware: **Apple M2 Pro, 12-core (8 performance + 4 efficiency)**. Times are for the single 8-D integral type above (9 sectors, 81 monomials, P linear), single-threaded unless noted; accuracy is the *median* over the 7200 sets and depends only on samples/sector M (batch-size independent).

operating point	Vegas (chunked `ncomp`)	plain MC
equal budget M=8000	**4.98% in 56 s**	7.85% in 50 s
equal budget M=24000	∼0.5%	5.46% in 270 s
time to ∼1% median	∼1.5 **min** / ∼10–15 **s (12 cores)**	∼1–2 h / ∼10–15 min (12 cores)

At *equal* sample budget MC and Vegas cost about the same wall time, but Vegas is ∼10× more accurate — so to hit a fixed ∼1% target Vegas needs far fewer samples and finishes ∼30–60× **sooner**. On the 12-core M2 Pro a 1%-accurate 7200-point n=8 scan is ∼10–15 **s with Vegas vs** ∼10–15 **min with MC**. Caveats: **(i)** absolute seconds drift ±1.5–2× with the laptop's thermal state (the M=24000 MC run clocked 270 s, ∼1.8× slower than a linear-in-M extrapolation) — the MC/Vegas *ratio* from one run is the robust quantity; **(ii)** MC's max error across the sets was **421%** (heavy tails, need lifting); Vegas held its max to ∼13–18%; **(iii)** QMC is no help at n=8 (worse than MC: 7.25% vs 5.46% at M=24000); **(iv)** both columns scale ∼linearly with #sectors × #monomials (the ratio holds); one-time preprocessing is seconds-to-a-minute, separate.

11 Limitations

Single-threaded; budgets capped at 10^5–10^6 samples/sector (enough to fix the ranking and rates, not tuned to a fixed high-precision target; MC/QMC parallelize trivially over kinematic points as the shipped code does). Convergence slopes are finite-range least-squares fits; for adaptive routines (Vegas) an apparent $p > 1$ reflects grid lock-on and should be read as "much faster than MC," not a literal asymptotic exponent. A thin-lattice-simplex workaround was required: the packaged fan code needs an integer interior point, which $\texttt{conv}\{0, e_i\}$ lacks for $n \geq 4$; since the normal fan is scale-invariant the fan is computed from $\texttt{conv}\{0, (n{+}2)e_i\}$ (validated against direct `NIntegrate`). All four CUBA routines and Sobol QMC agree with the exact reference (except Divonne), so the conclusions are not artifacts of one implementation.

Environment. macOS / Apple silicon; Apple clang 14 (`-std=c++17 -O3`); CUBA 4.2.2 and Boost 1.90 (Homebrew); Polymake 4.15; Wolfram Engine. Data: `TEST/INTERFILES/results.csv` (769 rows). Reproduce: `gen_sectors.wl` → `run.sh` → `analyze.py` → `plots.py`.